



node.js 기본

Backend Frameworks



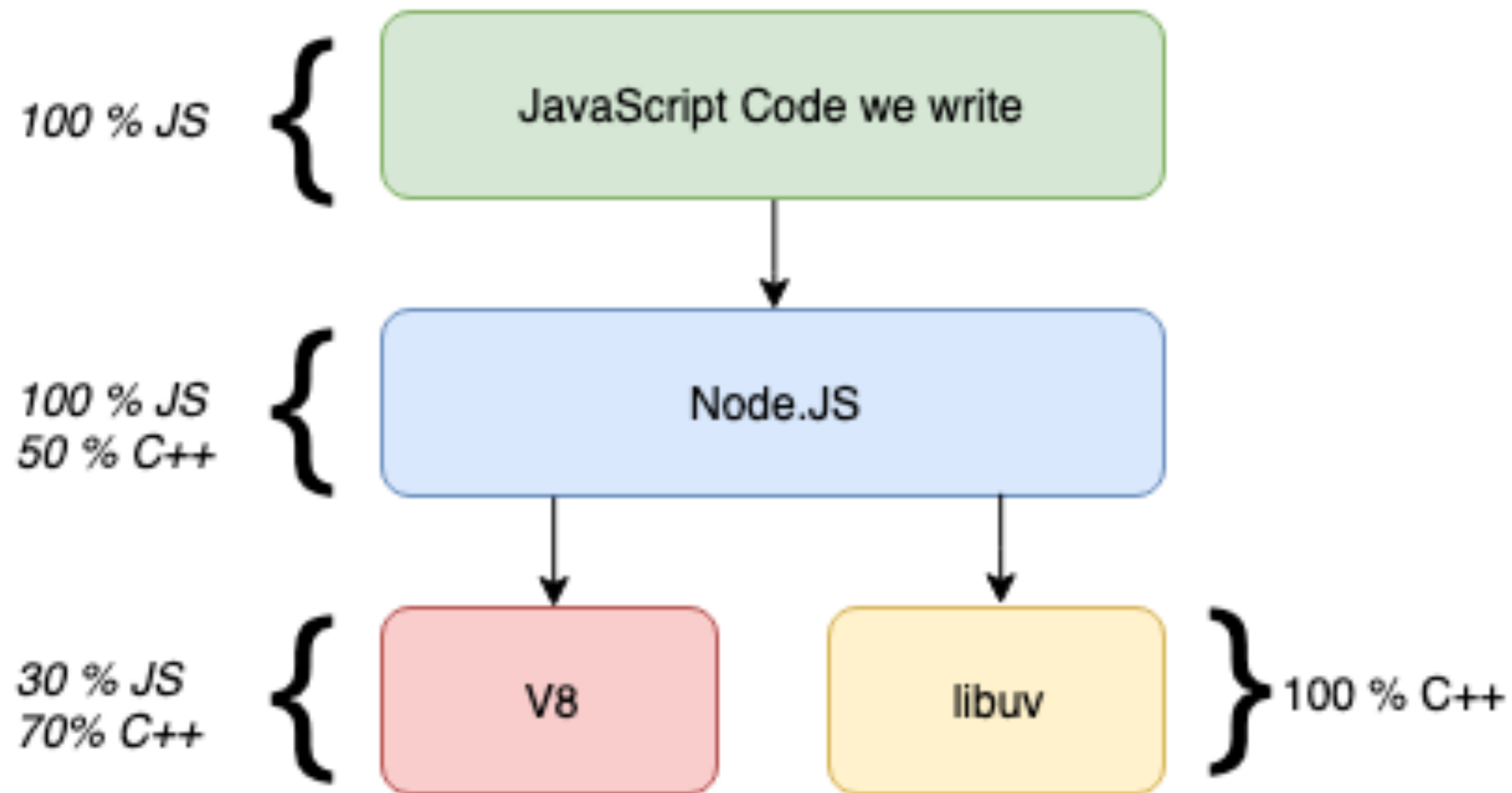
Node.js

- JavaScript Runtime 환경
- Ryan Dahl이 2009년에 출시
- Chrome의 JavaScript 엔진인 V8엔진 사용
- 다른 JavaScript Runtime 환경으로 디노(<https://deno.land>), 번(<https://bun.sh>) 등이 있음

Node.js

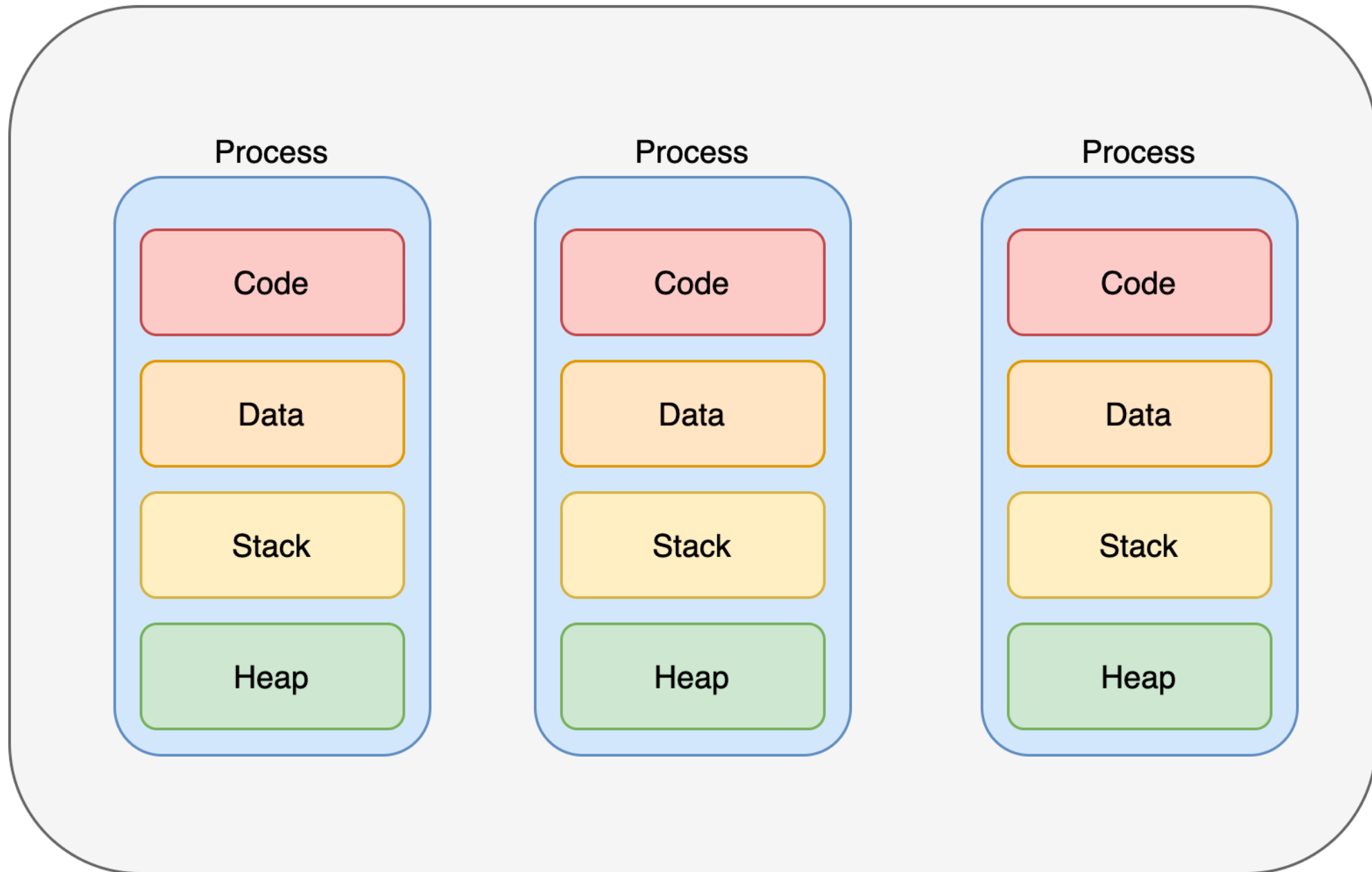
- 단일 스레드(Single Thread)
- 논 블로킹 이벤트 기반 방식
- 내장 HTTP 서버 라이브러리를 포함하고 있어 별도 서버 프로그램 필요없이 동작 가능
- 다양한 오픈 소스 라이브러리를 패키지 매니저를 통해서 쉽게 사용 가능(npm, yarn)

Node.js



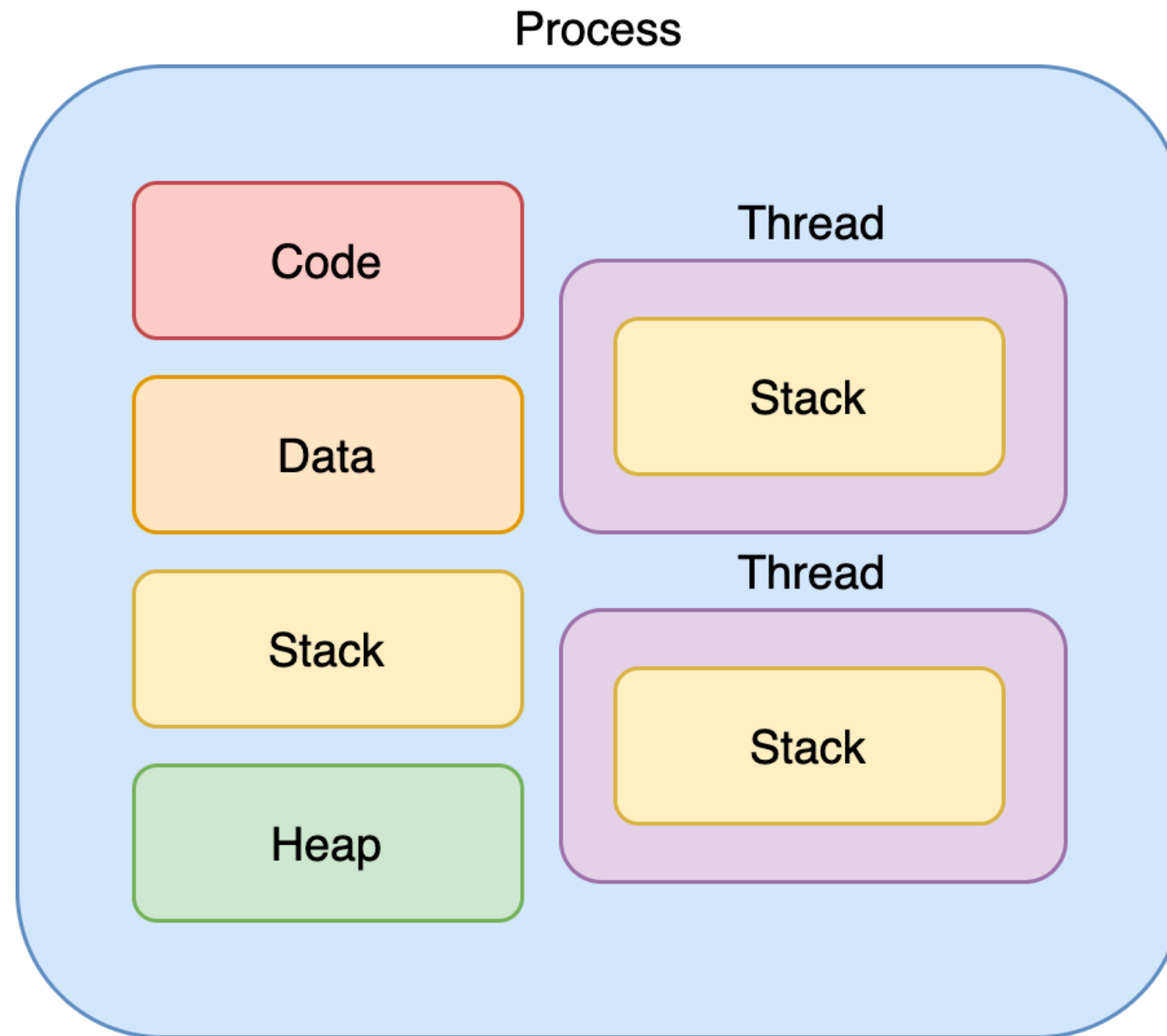
Process

운영체제(Operation System)



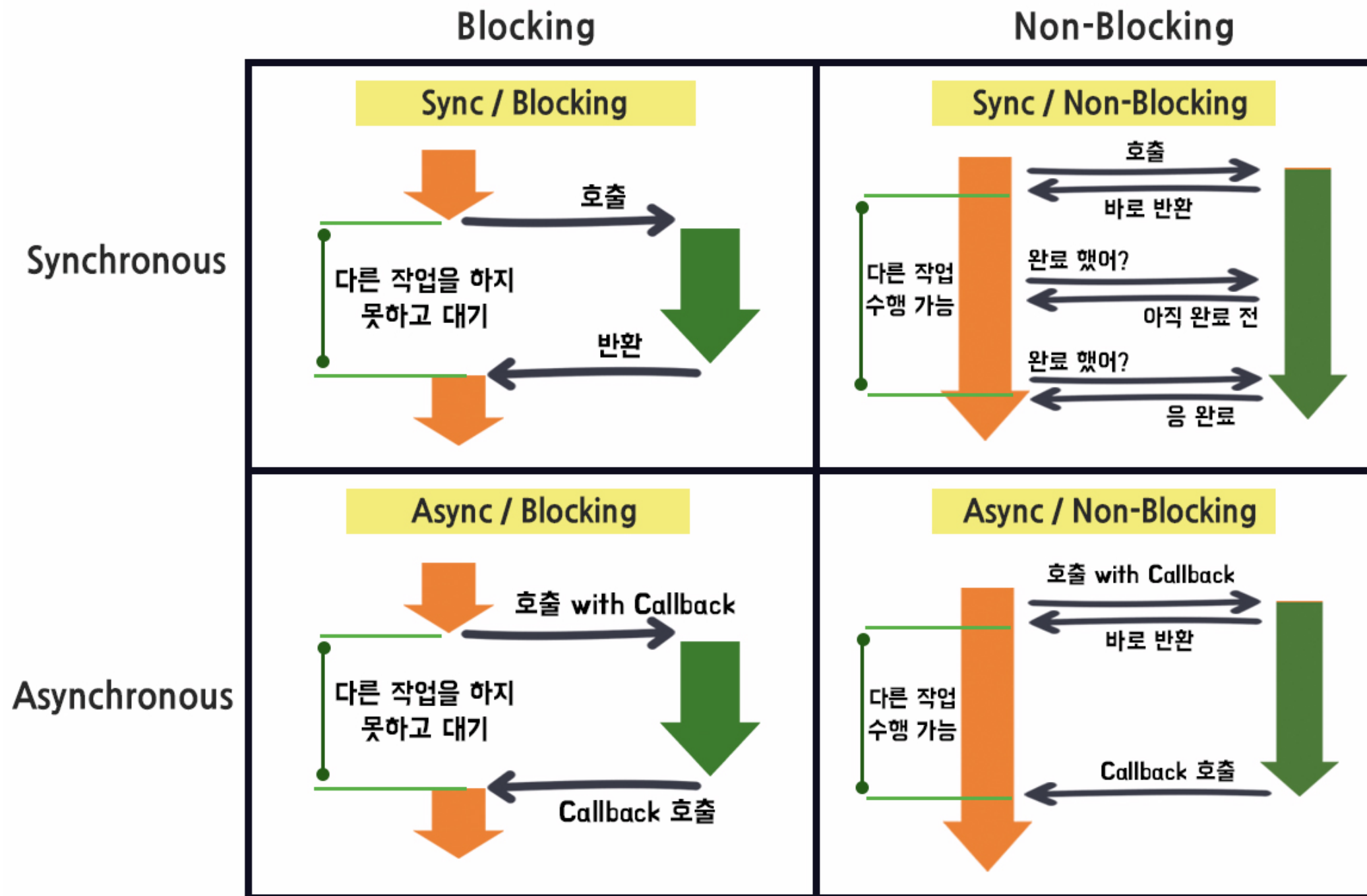
출처: 찰스의 안드로이드

Process



출처: 찰스의 안드로이드

sync/async, blocking/non-blocking



node 내장객체

global 객체

- 웹브라우저에서 window 같은 전역 객체
- node에서는 window나 document 객체 사용 불가
- 앞으로 사용할 console, process, timer 관련 함수들 가지고 있고 global은 생략 가능
- 웹브라우저와의 호환을 위해서 globalThis라는 객체를 사용

console

개발 중 디버깅을 위해 변수 내부 값 확인, 에러 내용 표시, 코드 실행 시간 표시 등을 위해 사용

```
globalThis.console.log('Hello node.js!!');  
console.info('Hello info');  
console.warn('Hello Warning');  
console.error('Hello error');
```

//시간 측정 시작

```
console.time();  
for (let i = 1; i < 100000; i++) {}
```

//시간 측정 종료

```
console.timeEnd();  
  
console.time('작업1');  
for (let i = 1; i < 100000; i++) {}  
console.timeEnd('작업1');
```

console

개발 중 디버깅을 위해 변수 내부 값 확인, 에러 내용 표시, 코드 실행 시간 표시 등을 위해 사용

```
// 서식문자를 사용한 치환
```

```
const person = {  
  name: '홍길동',  
  age: 18,  
  height: 180.5,  
};
```

```
console.log(  
  '이름은 %s, 나이는 %d살, 키는 %f, 객체는 %o',  
  person.name,  
  person.age,  
  person.height,  
  person  
);
```

```
console.table([person, person]);  
//객체의 속성 출력  
console.dir(person);
```

timer

타이머 기능을 제공하는 함수들

```
const timeout1 = setTimeout(function(){  
  console.log('1.5 초 후 한번 실행 ');  
}, 1500);
```

```
const timeout3 = setTimeout(function(){  
  console.log('3초 후 한번 실행 ');  
}, 3000);
```

```
const interval = setInterval(function(){  
  console.log('1초 인터벌 실행');  
}, 1000);
```

```
setTimeout(function(){  
  clearTimeout(timeout1);  
  clearTimeout(timeout3);  
  clearInterval(interval);  
  console.log('타이머 초기화');  
}, 2500);
```

```
setImmediate(function(){  
  console.log('즉시실행');  
});
```

process

현재 실행 중인 process와 시스템 정보와 관련된 속성과 함수

```
console.log(process.version);  
console.log(process.arch);  
console.log(process.platform);  
console.log(process.cwd());  
console.log(process.memoryUsage());  
console.log(process.uptime());  
console.log(process.env);  
process.exit();
```

__dirname, __filename

현재 실행되고있는 파일의 경로와 경로가 포함된 파일명

```
console.log(__dirname);
```

```
console.log(__filename);
```


node 내장모듈

Javascript Module import

- 모듈이란 변수나 함수 같은 코드가 모여있는 하나의 파일
- require 방식 (CommonJS)
 - Javascript를 브라우저 외부에서 사용 시 모듈화 작업을 할수 있게 CommonJS라는 워킹 그룹이 제안한 방식
 - CommonJS에서 모듈 명세를 정의
 - Node.js는 모듈 명세 1.0을 준수, Node.js 사용 시 기본
- import 방식(ES6 Module)
 - ES6 이후 Javascript 표준 방식
 - Node.js에서 사용하려면 확장자를 mjs를 사용 혹은 Package.json에서 "type": "module" 을 추가

OS

운영체제, CPU, 메모리 관련된 정보를 제공하는 모듈

```
const os = require('os');

console.log('<< 운영체제 정보 >>');
console.log('os.arch():', os.arch());
console.log('os.platform():', os.platform());
console.log('os.type():', os.type());
console.log('os.uptime():', os.uptime());
console.log('os.hostname():', os.hostname());
console.log('os.release():', os.release());
```

OS

운영체제, CPU, 메모리 관련된 정보를 제공하는 모듈

```
console.log('<< 경로 정보 >>');
```

```
console.log('os.homedir():', os.homedir());
```

```
console.log('os.tmpdir():', os.tmpdir());
```

```
console.log(process.cwd());
```

```
console.log('<< CPU 정보 >>');
```

```
console.log('os.cpus():', os.cpus());
```

```
console.log('os.cpus().length:', os.cpus().length);
```

```
console.log('<< Memory 정보 >>');
```

```
console.log('os.freemem():', os.freemem());
```

```
console.log('os.totalmem():', os.totalmem());
```

path

파일 및 폴더의 경로를 읽거나 만들때 필요한 기능을 제공하는 모듈

```
const path = require('path');
const filename = __filename;

console.log(filename);
console.log('path.sep:', path.sep);
console.log('path.delimiter:', path.delimiter);
console.log('path.dirname():', path.dirname(filename));
console.log('path.extname():', path.extname(filename));
console.log('path.basename():', path.basename(filename));
console.log(
  'path.basename - extname:',
  path.basename(filename, path.extname(filename))
);
```

path

파일 및 폴더의 경로를 읽거나 만들때 필요한 기능을 제공하는 모듈

```
console.log('path.parse()', path.parse(filename));
console.log('path.join()', path.join(__dirname, '..', 'kibwa'));
console.log(
  'path.resolve()',
  path.resolve(__dirname, '..', '/ios', 'kibwa')
);

console.log(path.posix.sep);
console.log('path.posix.join()', path.posix.join(__dirname, 'kibwa'));

console.log(path.win32.sep);
console.log('path.win32.join()', path.win32.join(__dirname, 'kibwa'));
```

fs

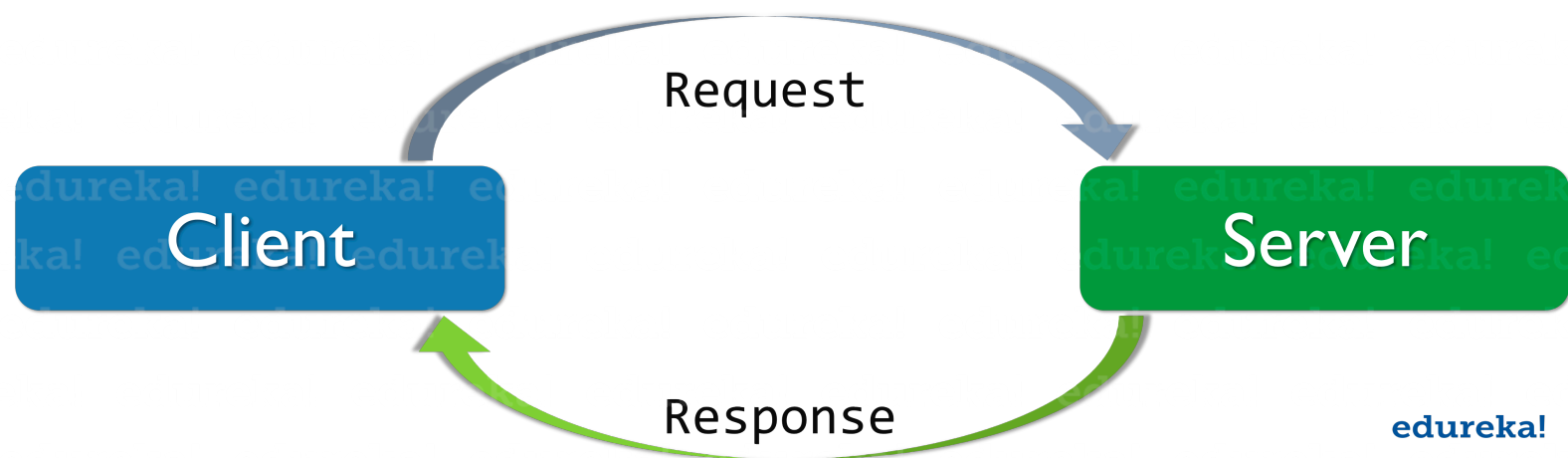
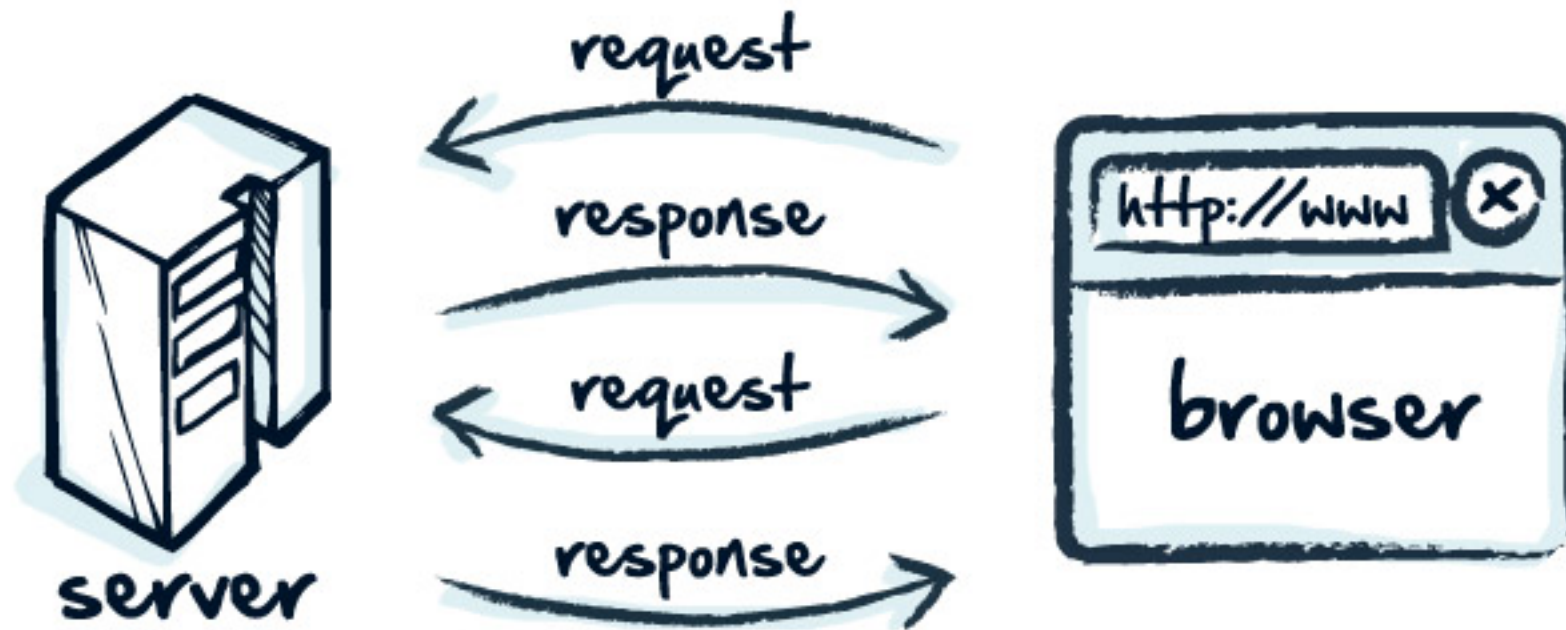
파일 생성, 읽기, 변경 및 삭제

```
const fs = require('fs');
```

```
fs.readFile('./test.txt', function(err, data){  
  if (err) {  
    throw err;  
  }  
  console.log(data);  
  console.log(data.toString());  
});
```

```
fs.writeFile('./test_write.txt', '파일쓰기 테스트입니다.', function(err){  
  if (err) {  
    throw err;  
  }  
  console.log('파일쓰기에 성공했습니다.');
```

Web Server



http

html 파일을 읽어서 서비스

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
                                   initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h1>서버가 동작중입니다.</h1>
  </body>
</html>
```


http

html 파일을 읽어서 서비스

```
const http = require('http');
const fs = require('fs');

const server = http.createServer(function (_, res) {
  fs.readFile('./index.html', function (_, data) {
    res.end(data);
  });
});

server.listen(3000, function () {
  console.log('Listening Server at 3000');
});
```

Http Status code

code	설명
1xx	요청을 받았으며 프로세스를 계속 진행
2xx	요청을 성공적으로 받았으며 인식했고 처리
3xx	요청 완료를 위해 추가 작업 조치가 필요
4xx	요청이 잘못되었거나 요청을 처리할 수 없음
5xx	서버의 오류로 요청을 처리할 수 없음

Http Method

code	설명
GET	리소스 읽기에 사용, 요청 데이터를 쿼리스트링을 통해서 전달
POST	데이터 등록에 사용, 요청 데이터를 바디에 담아서 전달
PUT	등록된 데이터의 대체에 사용, 요청 데이터를 바디에 담아서 전달
PATCH	등록된 데이터의 일부 수정에 사용, 요청 데이터를 바디에 담아서 전달
DELETE	등록된 데이터의 삭제에 사용

simple server

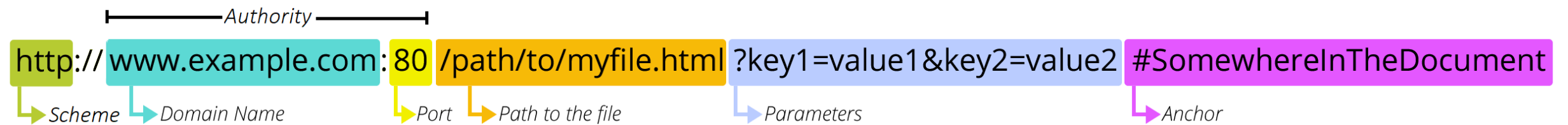
```
const http = require("http");  
const server = http.createServer((req, res) => {  
  res.setHeader("Content-Type", "text/html");  
  res.end("OK");  
});  
  
server.listen("3000", () => console.log("OK서버 시작!"));
```

http server

```
const http = require('http');
const fs = require('fs').promises;
const url = require('url');
```

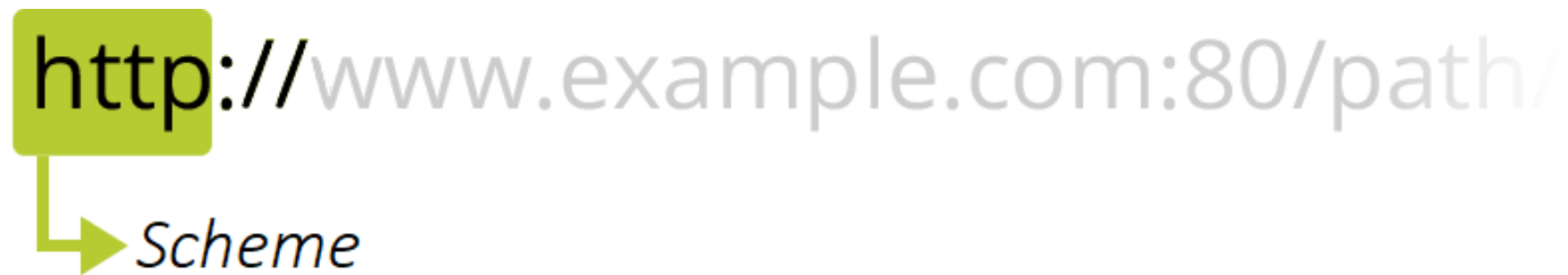
```
http
.createServer(async (req, res) => {
  var pathname = url.parse(req.url).pathname;
  if (req.method === 'GET') {
    if (pathname === '/') {
      const data = await fs.readFile('./index.html');
      res.end(data);
    } else if (pathname === '/person') {
      res.writeHead(200, {
        'Content-Type': 'application/json; charset=utf-8',
      });
      res.end(
        JSON.stringify({ name: '손흥민', age: 30, email: 'a@naver.com' })
      );
    }
  } else if (req.method === 'POST') {
    res.end(req.method);
  }
})
.listen(3000, function () {
  console.log('server running at 3000');
});
```

URL(Uniform Resource Locator)


The diagram shows the URL `http://www.example.com:80/path/to/myfile.html?key1=value1&key2=value2#SomewhereInTheDocument` with its components highlighted in colored boxes and labeled with arrows:

- http://** (green box) is labeled *Scheme*.
- www.example.com** (cyan box) is labeled *Domain Name*.
- :80** (yellow box) is labeled *Port*.
- /path/to/myfile.html** (orange box) is labeled *Path to the file*.
- ?key1=value1&key2=value2** (light blue box) is labeled *Parameters*.
- #SomewhereInTheDocument** (purple box) is labeled *Anchor*.

A bracket above the `www.example.com:80` portion is labeled *Authority*.


The diagram shows the `http://` portion of a URL highlighted in a green box, with an arrow pointing to it labeled *Scheme*.


The diagram shows the `www.example.com:80` portion of a URL. The `www.example.com` part is highlighted in a cyan box and labeled *Domain Name*, while the `:80` part is highlighted in a yellow box and labeled *Port*.

`https://developer.mozilla.org/`

URL(Uniform Resource Locator)

m:80/path/to/myfile.html?key1=value1&

→ *Path to resource*

html?key1=value1&key2=value2#Some

→ *Parameters*

lue2#SomewhereInTheDocument

→ *Anchor*

<https://developer.mozilla.org/>

URL(Uniform Resource Locator)

```
const strURL =
```

```
    'http://wizard113:password@apis.womanup.co.kr:8080/path1/path2?  
serviceKey=key&base_date=20240407&dataType=JSON&numberOfRows=10&p  
ageNo=1&category=api&category=geo';
```

```
const urls = new URL(strURL);
```

```
console.log('protocol: ', urls.protocol); // 'http:'
```

```
console.log('host: ', urls.host); // 'apis.womanup.co.kr:8080'
```

```
console.log('hostname: ', urls.hostname); // 'apis.womanup.co.kr'
```

```
console.log('port: ', urls.port); // '8080'
```

```
console.log('username: ', urls.username); // 'wizard113'
```

```
console.log('password: ', urls.password); // 'password'
```

```
console.log('pathname: ', urls.pathname); // '/path1/path2'
```

```
console.log('search: ', urls.search); // '?serviceKey=key&...'
```

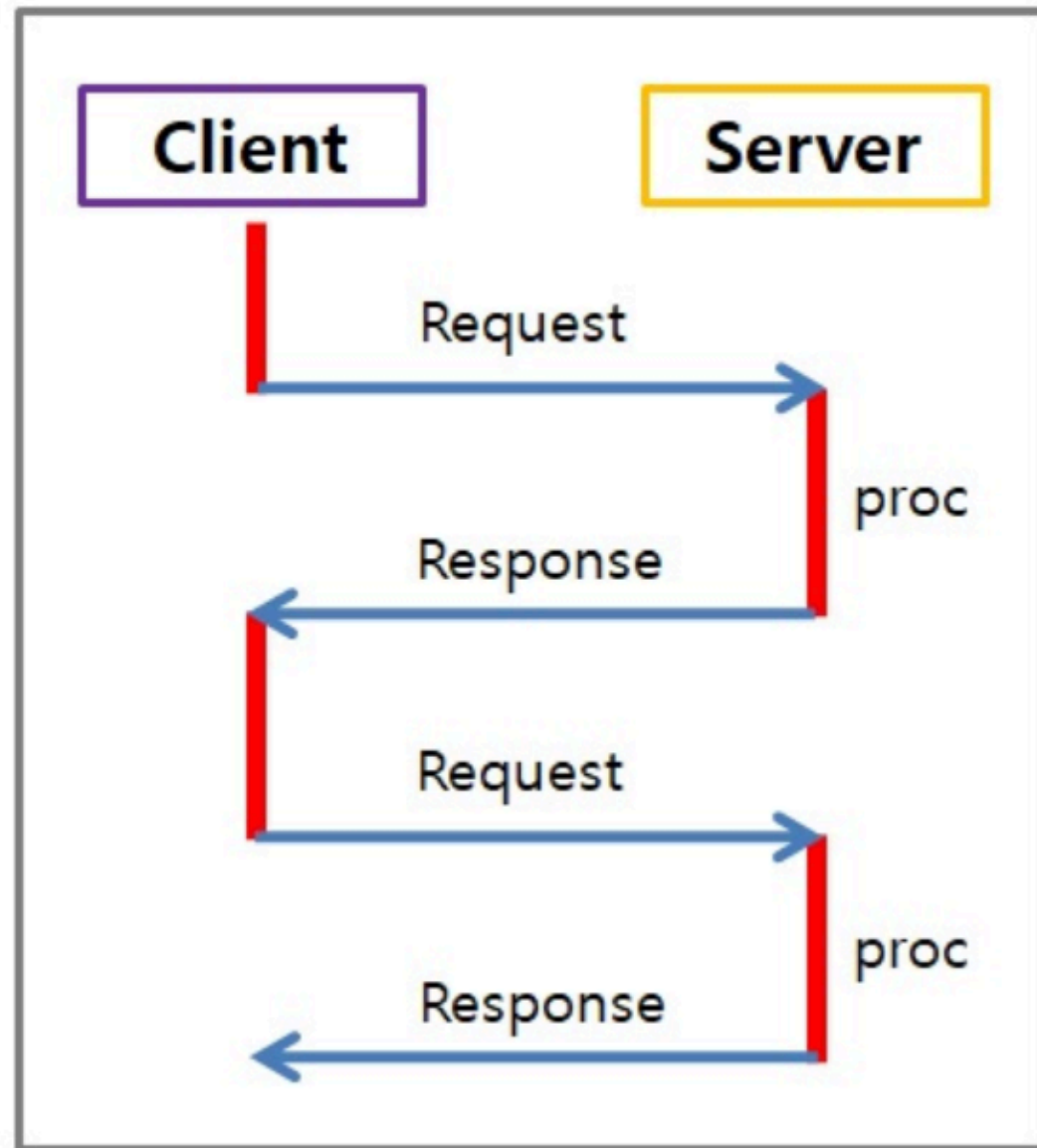
```
console.log('category 전체: ',
```

```
urls.searchParams.getAll('category')); // ['api', 'geo']
```


JavaScript 비동기 프로그래밍

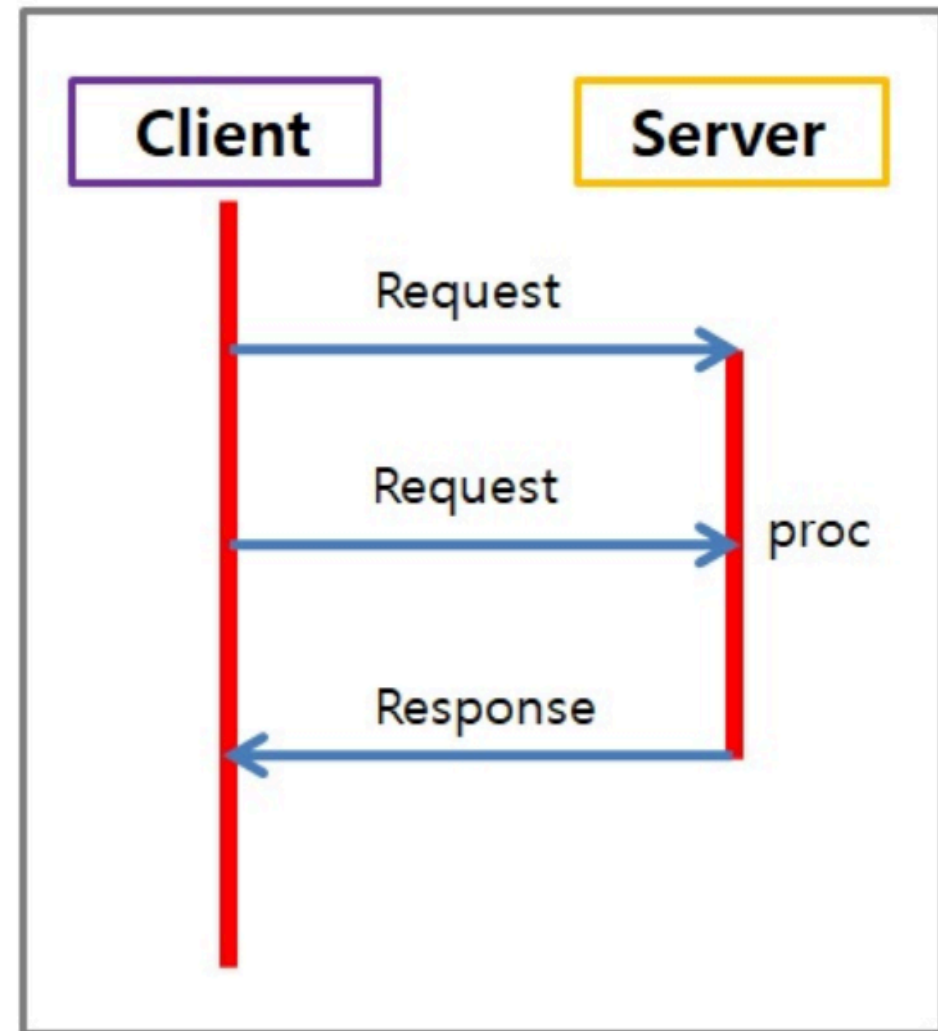
동기 프로그래밍

```
console.log('1번');  
console.log('2번');  
console.log('3번');
```



비동기 프로그래밍

```
console.log('1번');  
setTimeout(() => {  
  console.log('2번');  
}, 1000);  
console.log('3번');
```



- setTimeout
 - 특정 작업을 일정 시간이 지난 후에 실행하도록 예약하는 비동기 함수
 - n 밀리초(ms) 후에 전달된 콜백을 실행 시킵니다.

비동기 프로그래밍

```
function asyncTest(name, callback) {  
  console.log('타이머 시작');  
  setTimeout(() => {  
    callback(name);  
  }, 3000); // 3초 후에 데이터를 가져온다고 가정  
}
```

```
function doOtherthing() {  
  for (let i = 0; i < 300; i++) {  
    console.log(`${i}번째 처리`);  
  }  
}  
  
asyncTest('뷔', sayHello);  
doOtherthing();
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
const DB = [];
```

```
function save2DB(user, callback) {  
  DB.push(user);  
  console.log(`${user.name}님 데이터베이스에 저장 완료되었습니다.`);  
  return callback(user);  
}
```

```
function sendEmail(user, callback) {  
  console.log(`${user.email}으로 이메일이 전송 완료되었습니다.`);  
  return callback(user);  
}
```

```
function getResult(user) {  
  return `${user.name}님 회원가입에 성공했습니다.`;  
}
```

callback 지옥

콜백 함수가 반복되어 코드의 가독성이 떨어짐

```
function register(user) {  
    return save2DB(user, (user) => {  
        return sendEmail(user, (user) => {  
            return getResult(user);  
        });  
    });  
}
```

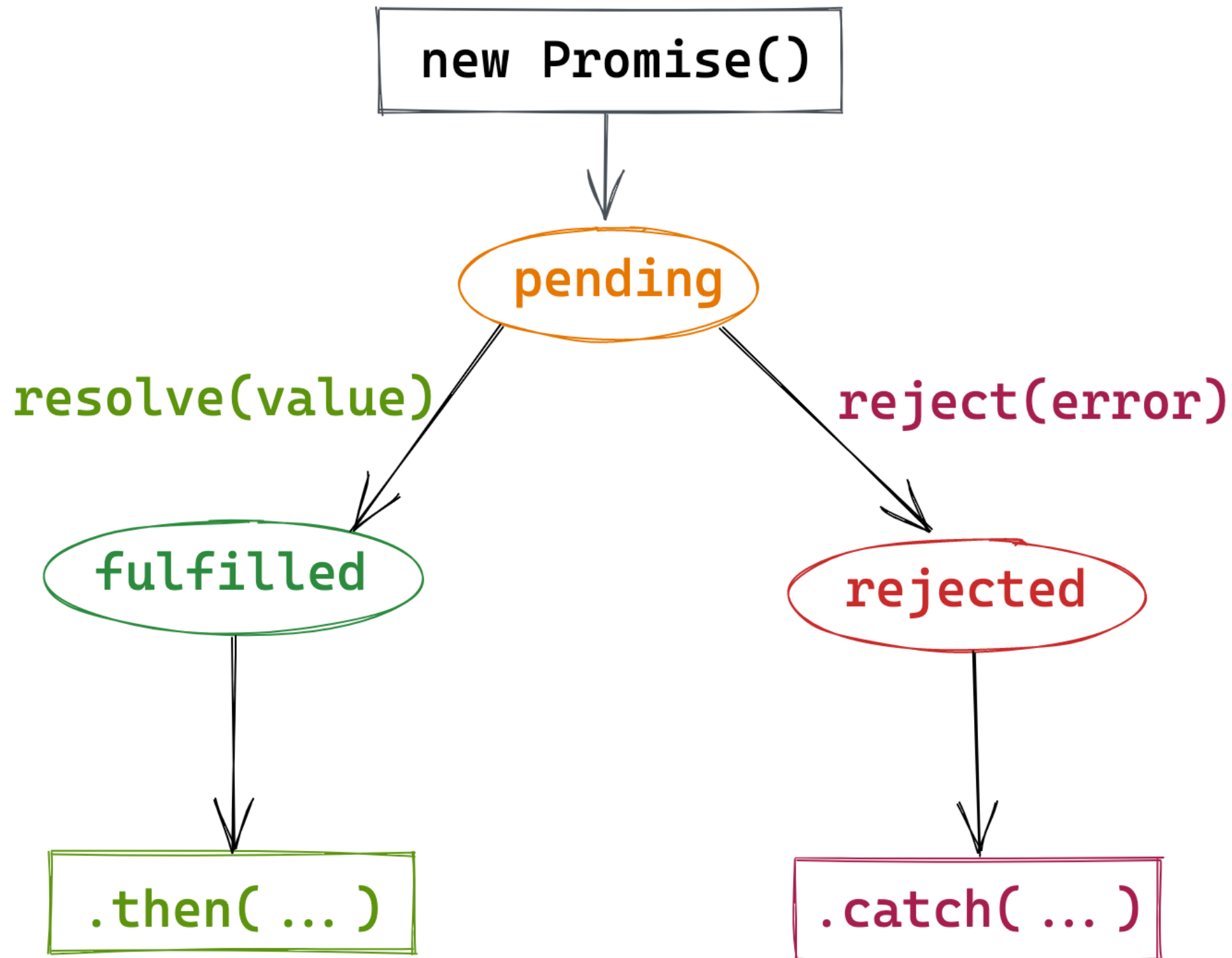
```
const result =
    register({ name: '손흥민', email: 'son@naver.com' });
console.log(result);
```



Promise

- 콜백의 단점을 보완하기 위해 만들어진 비동기 처리 객체
- 비동기 처리 시점을 명확히 표현 가능
- 연속된 비동기 처리 작업을 처리하기 간편
- Promise는 3개의 상태를 가짐
 - pending: 아직 실행되지 않은 초기 상태
 - fulfilled: 연산이 성공적으로 완료됨
 - rejected: 연산이 실패함

Promise



Promise

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve('작업 성공!');  
  } else {  
    reject('작업 실패!');  
  }  
});  
promise  
  .then((result) => {  
    console.log('성공 결과:', result);  
  })  
  .catch((error) => {  
    console.error('실패 결과:', error);  
  });
```

Promise

```
const p = new Promise((resolve) => {  
  resolve(10);  
  console.log('1. Promise 실행');  
});
```

```
console.log('2. 코드 계속 실행');
```

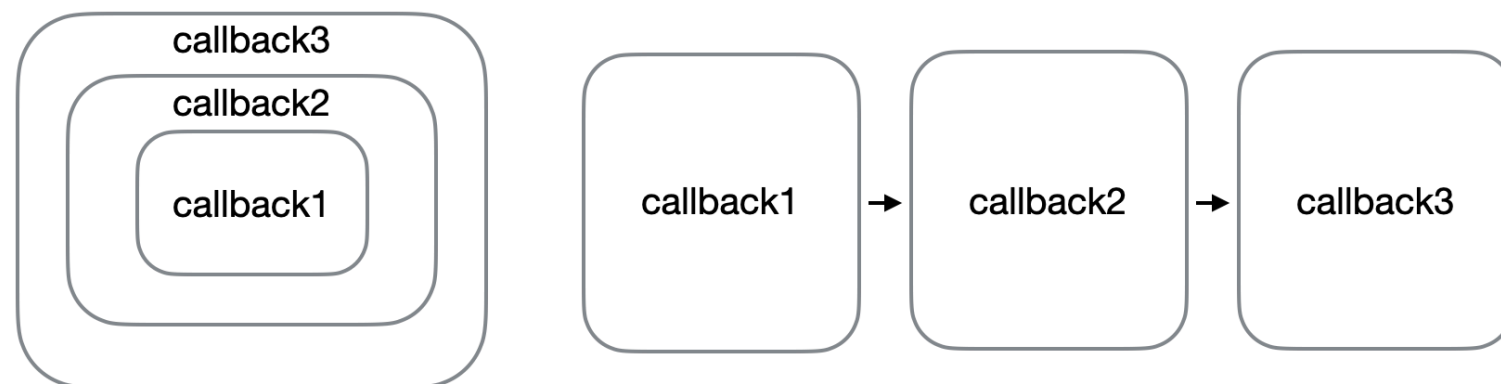
```
p.then((num) => {  
  console.log('3. then 실행:', num);  
});
```

Promise-체이닝

```
const p1 = new Promise((resolve) => {  
  const result = 10;  
  resolve(result);  
});
```

//반환값:11 => resolve(11)을 호출하는 Promise를 생성

```
const p2 = p1.then((num) => num + 1);  
p2.then((num) => console.log(num));  
p1.then((num) => num + 1).then((num) => console.log(num));
```



즉시 성공 Promise

```
new Promise((resolve) => {  
    resolve(10);  
});  
Promise.resolve(10);  
//함수가 항상 Promise를 반환하게 만들기  
function getData() {  
    return Promise.resolve('데이터');  
}  
getData().then(console.log);  
//Promise 체이닝을 사용  
Promise.resolve(1)  
    .then((n) => n + 1)  
    .then(console.log);
```

즉시 실패 Promise

```
Promise.reject('에러').catch(console.log);
```

```
function checkAge(age) {  
  if (age < 18) {  
    return Promise.reject('미성년자');  
  }  
  return Promise.resolve('통과');  
}
```

```
checkAge(25).then(console.log).catch(console.log);
```

Promise - 회원가입

```
const DB = [];  
  
function saveDB(user) {  
  const oldDBLength = DB.length;  
  DB.push(user);  
  console.log(`${user.username} 저장 완료되었습니다.`);  
  return new Promise((resolve, reject) => {  
    if (DB.length > oldDBLength) {  
      resolve(user);  
    } else {  
      reject(new Error('저장에 실패했습니다.!' ));  
    }  
  });  
}
```

Promise - 회원가입

```
function sendEmail(user) {  
  console.log(`${user.email}으로 이메일을 발송했습니다.`);  
  return new Promise((resolve) => {  
    resolve(user);  
  });  
}
```

```
function getResult(user) {  
  return new Promise((resolve) => {  
    resolve(`${user.username}님 등록 성공했습니다.`);  
  });  
}
```


Promise - 회원가입

```
function registerByPromise(user) {  
  const result = saveDB(user)  
    .then(sendEmail)  
    .then(getResult)  
    .catch((error) => new Error(error));  
  return result;  
}
```

```
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
const result = registerByPromise(myUser);  
result.then(console.log);
```


Promise - 영화정보

```
const url =  
  'https://raw.githubusercontent.com/wizard113/datas/main/  
movieinfo.json';  
  
fetch(url)  
  .then((response) => {  
    return response.json();  
  })  
  .catch(() => {  
    console.log('요청에 실패했습니다. ');  
  })
```

Promise - 영화정보

=> 앞에서 계속

```
.then((data) => {  
    if (!data) {  
        throw new Error('데이터가 없습니다.');    }  
    if (!data.articleList || data.articleList.length === 0) {  
        throw new Error('데이터가 없습니다.');    }  
    return data.articleList;  
})  
.catch((error) => {  
    console.error('에러 발생:', error.message);  
})
```

Promise - 영화정보

=> 앞에서 계속

```
.then((articles) => {  
    return articles.map((article, idx) => {  
        return { title: article.title, rank: idx + 1 };  
    });  
})  
.then((results) => {  
    for (let movie of results) {  
        console.log(`[${movie.rank}위] ${movie.title}`);  
    }  
})  
.catch((err) => {  
    console.log('<<에러발생>>');  
    console.log(err);  
});
```

async/await

- Promise를 더 쉽게 다루기 위한 문법
- 비동기 코드를 동기 코드처럼 깔끔하게 작성 가능
- async function은 자동으로 Promise로 감싸서 반환
- await는 Promise가 해결될 때까지 대기
- await는 반드시 async 함수 안에서만 사용 가능
- await 오른쪽은 반드시 Promise

async/await

// async 키워드가 붙은 함수는 반환값을 Promise로 감싸서 반환

```
async function func1() {  
  return 'hello';  
}
```

```
func1().then(console.log);
```

```
function func2() {  
  return new Promise((resolve) => {  
    resolve('hello');  
  });  
}
```

```
func2().then(console.log);
```

async/await

```
async function func3() {  
  //await는 뒤에 오는 프로미스가 실행완료될때까지 기다림  
  //async함수에서만 사용가능  
  let name = await func1();  
  console.log(name);  
}  
  
func3();
```

async/await - 회원가입

```
async function registerByAsync(user) {  
  try {  
    const savedUser = await saveDB(user);  
    const emailedUser = await sendEmail(savedUser);  
    const result = await getResult(emailedUser);  
    return result;  
  } catch (error) {  
    return new Error(error);  
  }  
}  
  
const myUser = { uname: '손흥민', email: 'son@naver.com' };  
registerByAsync(myUser).then(console.log);
```


async/await - 영화정보

// 1. 데이터 가져오기(fetch)

```
async function fetchMovieData(url) {  
  const response = await fetch(url);  
  if (!response.ok) {  
    throw new Error('요청에 실패. 상태 코드: ' + response.status);  
  }  
  const data = await response.json();  
  return data;  
}
```

// 2. 데이터 검증

```
function validateMovieData(data) {  
  if (!data.articleList || data.articleList.length === 0) {  
    throw new Error('데이터가 없습니다. ');  
  }  
}
```


async/await - 영화정보

// 3. 영화 정보 추출

```
function extractMovieInfos(articleList) {  
  return articleList.map((article, idx) => ({  
    title: article.title,  
    rank: idx + 1,  
  }));  
}
```

// 4. 영화 출력

```
function displayMovies(movieInfos) {  
  for (const movie of movieInfos) {  
    console.log(`[${movie.rank}위] ${movie.title}`);  
  }  
}
```

async/await - 영화정보

```
const url = 'http://raw.githubusercontent.com/wizard113/  
            datas/main/movieinfo.json';
```

```
async function movies() {  
  try {  
    const data = await fetchMovieData(url);  
    validateMovieData(data);  
    const movieInfos = extractMovieInfos(data.articleList);  
    displayMovies(movieInfos);  
  } catch (err) {  
    console.error('에러 발생:', err.message);  
  }  
}  
  
movies();
```



- npm(Node Package Manager)은 JavaScript 및 세계 최대의 소프트웨어 레지스트리 패키지 관리자
- npm registry에는 Node.js에서 사용되는 각종 패키지들이 존재
- npm을 사용해서 필요한 라이브러리를 설치
- Package.json, Package-lock.json으로 의존성 관리
- npm init 으로 초기화

Package.json

항목	내용
package name	패키지 이름으로 프로젝트 혹은 현재 애플리케이션 이름을 말한다.
version	패키지, 프로젝트, 애플리케이션 버전 면을 말한다.
description	패키지, 프로젝트, 애플리케이션의 설명을 말한다.
entry point	프로젝트에서 가장 먼저 실행되는 자바스크립트 실행파일로 웹 서버의 경우 index.js 혹은 app.js 로 지정해 준다.
test command	코드 테스트 시에 입력해 줄 명령어로, 사용하지 않을 경우 빈 값으로 지정된다.
git repository	코드를 저장해둔 깃(Git) 저장소 주소로 사용하지 않을 경우 빈 값으로 지정된다.
keywords	패키지, 프로젝트, 애플리케이션의 검색을 위한 키워드
license	패키지, 프로젝트, 애플리케이션의 라이선스로 기본값은 ISC (Internert Systems Consortium)에서 허용한 자유 소프트웨어 라이선스



- **Node.js**를 위한 빠르고 개방적인 간결한 웹 프레임워크
- 사실상 Nodejs의 표준 웹서버 프레임워크로 불려질 만큼 많은 곳에서 사용
- 웹 애플리케이션을 만들기 위한 각종 라이브러리와 미들웨어 등이 내장

Express를 이용한 server

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.json({
    success: true,
  });
});

app.listen(port, () => {
  console.log(`server is listening at localhost:${port}`);
});
```

express를 이용한 server

```
const express = require('express');
const app = express();
const port = 3000;
let person = [];

app.use(express.json());
//요청을 application/json으로 보냈을때 파싱해서 req.body에 담아줌
app.get('/', (req, res) => {
  res.send('Hello Express!');
});
app.get('/person', (req, res) => {
  res.json(person);
});
app.post('/person', (req, res) => {
  person.push(req.body);
  res.json(req.body);
});

app.listen(port, () => {
  console.log(`server is listening at localhost:${port}`);
});
```


Express를 이용한 server

res.send(arg), res.json(arg), res.end()

res.send(arg)	send에 전해진 argument의 종류에 따라 Content-type이 자동적으로 설정
res.json(arg)	arg가 json이 아닌 것도 json 형식전송, content-type 헤더를 application/JSON 설정, 마지막에는 자체적으로 res.send()호출
res.end()	보내줄 아무 데이터도 없는데 response를 끝내고 싶을 때 사용한다.

express를 이용한 server

```
const express = require('express');
const app = express();
let posts = [];
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true })); //qs 모듈을 사용

app.get('/', (_, res) => {
  res.json(posts);
});

app.post('/posts', (req, res) => {
  const { title, name, text } = req.body;
  const post = { id: posts.length + 1,
                 title, name, text, created: Date() };
  posts.push(post);
  res.json(post);
});
```

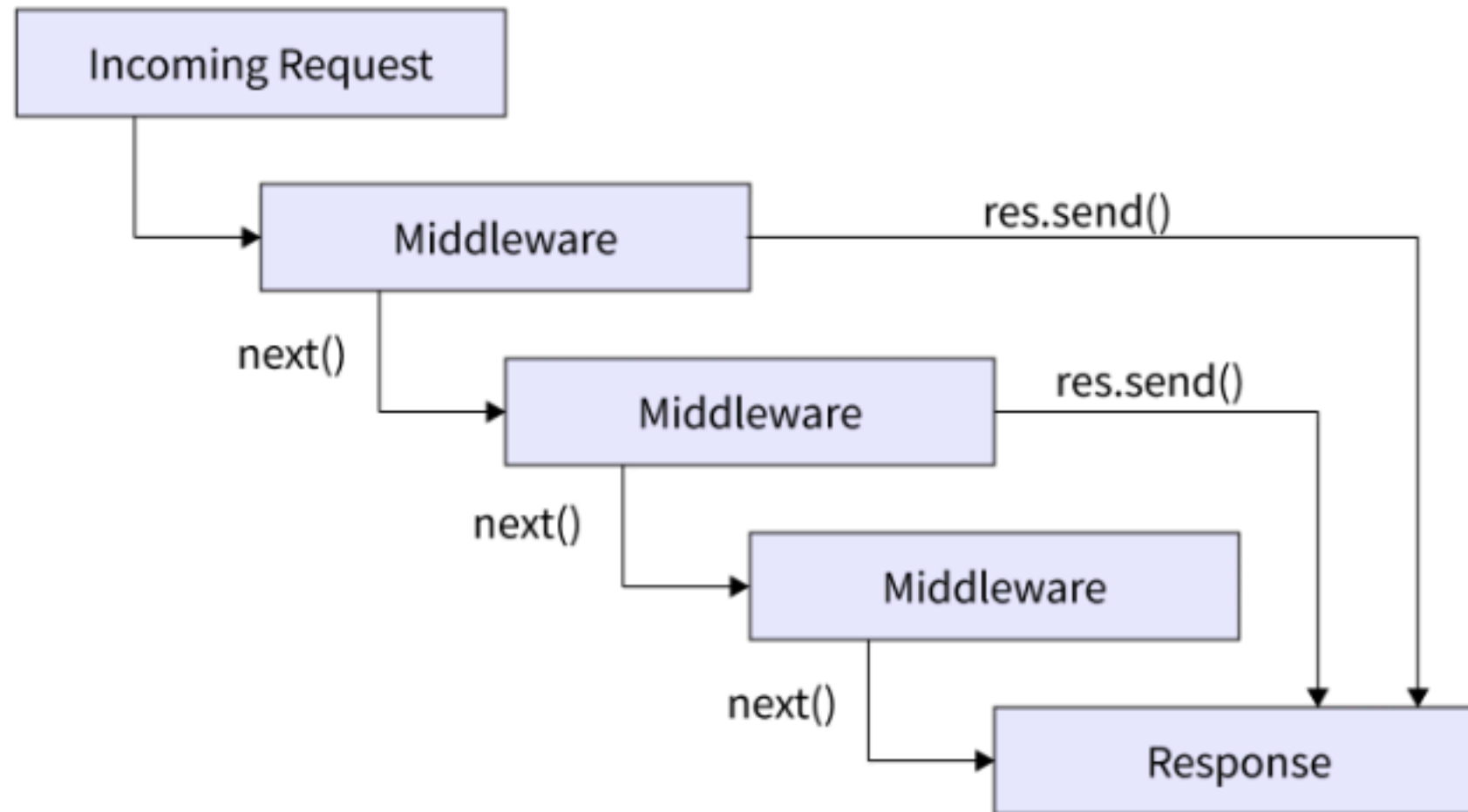
express를 이용한 server

```
app.delete('/posts/:id', (req, res) => {  
  const id = req.params.id;  
  let filteredPosts = posts.filter((post) => post.id !== +id);  
  let isLengthChanged = filteredPosts.length !== posts.length;  
  posts = filteredPosts;  
  if (isLengthChanged) {  
    res.json('OK');  
    return;  
  }  
  res.json('Delete Failed');  
});  
  
app.listen(port, () => {  
  console.log('post server Start!');  
});
```

express의 middleware

- 미들웨어는 HTTP 요청발생과 응답완료 사이에 동작하는 함수
- HTTP 요청이 발생하면 순차적으로 미들웨어 실행
- HTTP 응답이 완료될때까지 req 및 res 객체를 조작하거나 다음 미들웨어를 실행하기 위해 next()를 호출
- 미들웨어는 req, res, next 3개의 매개변수를 가짐

express middleware



express의 middleware

```
var express = require('express');  
var app = express();
```

```
app.get('/', function(req, res, next) {  
  next();  
})
```

```
app.listen(3000);
```

미들웨어 함수가 적용되는 HTTP 메소드.

미들웨어 함수가 적용되는 경로(라우트).

미들웨어 함수.

미들웨어 함수에 대한 콜백 인수(일반적으로 "next"라 불림).

미들웨어 함수에 대한 HTTP 응답 인수(일반적으로 "res"라 불림).

미들웨어 함수에 대한 HTTP 요청 인수(일반적으로 "req"라 불림).

express의 middleware

메소드	동작
app.use(middleware)	모든 요청에서 해당 미들웨어 실행
app.use('path', middleware)	path 하위의 모든 요청(get, post, ..)에서 미들웨어 실행
app.all('path', middleware)	path와 일치 하는 모든 요청(get, post, ..)에서 미들웨어 실행
app.get('path', middleware)	path와 일치 하는 get 요청에서 미들웨어 실행
app.post('path', middleware)	path와 일치 하는 post 요청에서 미들웨어 실행
app.put('path', middleware)	path와 일치 하는 put 요청에서 미들웨어 실행
app.delete('path', middleware)	path와 일치 하는 delete 요청에서 미들웨어 실행

express의 middleware

```
const express = require('express');  
const app = express();  
const port = 3000;
```

```
app.use((req, res, next) => {  
  console.log('app.use()');  
  next();  
});
```

```
var requestTime = function (req, res, next) {  
  req.requestTime = Date.now();  
  next();  
};  
app.use(requestTime);
```

```
app.use('/', (req, res, next) => {  
  console.log('app.use(/)');  
  next();  
});
```

=> 다음 페이지에서 계속

express의 middleware

```
app.all('/', (req, res, next) => {  
  console.log('app.all(/)');  
  next();  
});
```

```
app.all('/about', (req, res, next) => {  
  console.log('app.all(/about)');  
  next();  
});
```

```
app.get('/', (req, res) => {  
  console.log('app.get(/)');  
  res.send('OK');  
});
```

```
app.get('/about', (req, res) => {  
  console.log('app.get(/about)');  
  res.send('OK');  
});
```

=> 다음 페이지에서 계속

express middleware

```
app.post('/', (req, res) => {  
  console.log('app.post(/)');  
  res.send('OK');  
});
```

```
app.put('/', (req, res) => {  
  console.log('app.put(/)');  
  res.send('OK');  
});
```

```
app.delete('/', (req, res) => {  
  console.log('app.delete(/)');  
  res.send('OK');  
});
```

```
app.listen(port, () => {  
  console.log(`Server start at ${port}`);  
});
```

express의 자주 사용하는 middleware

morgan: 요청과 응답에 대한 추가적인 로그를 표시

```
const express = require('express');  
const app = express();  
const morgan = require('morgan');  
const port = 3000;
```

```
app.use(morgan('dev'));  
//dev, combined, common, short, tiny 등의 인자가 올수있다.
```

```
app.get('/', (req, res) => {  
  res.send('OK');  
});
```

```
app.listen(port, () => {  
  console.log(`Server start at ${port}`);  
});
```

GET / 200 3.773 ms - 2

POST /posts 404 3.505 ms - 145

요청메소드, 경로, 응답의 상태코드, 처리시간, 전송데이터의 크기

npm install morgan

express의 자주 사용하는 middleware

dotenv: .env파일의 내용을 process.env로 설정해줌

```
//.env  
COOKIE_SECRET = 1234  
PORT = 3000  
DB_PASSWORD = 5678
```

```
const dotenv = require('dotenv');  
//실행시 현재폴더의 .env파일을 읽어 process.env에 설정  
dotenv.config();  
//.env가 다른경로에 있을때는 path 옵션으로 넘기면됨  
// dotenv.config({ path: './path/.env' });  
console.log(process.env.COOKIE_SECRET);  
console.log(process.env.DB_PASSWORD);  
console.log(process.env.PORT);
```

npm install dotenv

express의 자주 사용하는 middleware

body-parser: 요청의 body에 있는 데이터를 해석해서 req.body 객체를 만들어줌

```
const express = require('express');
const app = express();
const bodyParser = require('body-parser');
const port = 3000;

//요청을 application/json으로 보냈을때 파싱해서 req.body에 담아줌
app.use(bodyParser.json());
//요청을 application/x-www-form-urlencoded로 보냈을때 파싱해서 req.body에 담아줌
app.use(bodyParser.urlencoded({ extended: true }));
app.post('/', (req, res) => {
  console.log('req.body : ', req.body);
  console.log('name : ', req.body.name);
  res.send(req.body);
});

app.listen(port, () => {
  console.log(`Server start at ${port}`);
});
```

npm install body-parser

express의 자주 사용하는 middleware

multer: 요청을 multipart/form-data로 보낼때 사용하는 미들웨어

```
const multer = require('multer');
const express = require('express');
const morgan = require('morgan');
const path = require('path');
const fs = require('fs');
const app = express();
const port = 3000;

try {
  fs.readdirSync('files');
} catch (error) {
  console.error('files 폴더가 새로 만듬');
  fs.mkdirSync('files');
}
```

express의 자주 사용하는 middleware

multer: 요청을 multipart/form-data로 보낼때 사용하는 미들웨어

```
const upload = multer({ //multer 객체생성
  storage: multer.diskStorage({
    destination(req, file, done) {
      done(null, 'files/');
    },
    filename(req, file, done) {
      const ext = path.extname(file.originalname);
      done(null, path.basename(file.originalname, ext) +
        Date.now() + ext);
    },
  }),
  limits: { fileSize: 10 * 1024 * 1024 },
});
```

express의 자주 사용하는 middleware

multer: 요청을 multipart/form-data로 보낼때 사용하는 미들웨어

```
app.post('/upload', upload.single('image'), //하나의 파일 전송
  (req, res) => {
    console.log(req.file);
    console.log(req.body.email);
    res.send('ok');
  });
```

```
app.post(
  '/uploadfiles', //여러파일을 각각의 키로 전송
  upload.fields([ { name: 'image' }, { name: 'file' } ]),
  (req, res) => {
    console.log(req.files);
    console.log(req.body.email);
    res.send('ok');
  }
);
```


express의 자주 사용하는 middleware

multer: 요청을 multipart/form-data로 보낼때 사용하는 미들웨어

```
app.post('/uploadimages', //여러파일을 하나의 키로 전송
  upload.array('image'), (req, res) => {
    console.log(req.files);
    console.log(req.body);
    res.send('ok');
  });

app.listen(port, () => {
  console.log(`Server start at ${port}`);
});
```


response로 파일을 보내는 법

```
app.get('/image', (req, res) => {  
  const filename = req.query.filename;  
  return res.sendFile(process.cwd() + '/files/' + filename);  
});
```

express의 querystring 파싱

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  const query = req.query;
  console.log(query);
  console.log(`이름은 ${query.name} 이고 나이는 ${query.age}`);
  res.send(query);
});

app.listen(port, () => {
  console.log(`Server start at ${port}`);
});
```

express Router 활용

메소드와 경로별로 처리할 경우 코드의 가독성이 떨어지고 유지보수가 불편

```
const express = require('express');
const app = express();
const port = 3000;

const mainRouter = require('./routes'); //index.js 파일명 생략가능
const studentRouter = require('./routes/student');

app.use('/', mainRouter);
app.use('/student', studentRouter);

app.listen(port, () => {
  console.log(`Server start at ${port}`);
});
```

express Router 활용

router용 모듈을 따로 만듦 - routes/index.js

```
const express = require('express');  
const router = express.Router();
```

//path는 기존경로에 현재경로가 합쳐져서 라우팅

```
router.get('/', (req, res) => {  
  res.send('Hello, index');  
});
```

```
module.exports = router;
```

express Router 활용

router용 모듈을 따로 만듦 - routes/student.js

```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('Hello, student!');
});

router.get('/admin', (req, res) => {
  res.send(`Hello admin student `);
});

router.get('/:name', (req, res) => {
  //param이 있을때는 순서에 유의
  res.send(`안녕 ${req.params.name} 학생! `);
});
module.exports = router;
```

{ REST : API }

Representational State Transfer API

REST API의 탄생

- 로이 필딩 박사의 학위 논문(2000년)에서 최초로 소개
- 로이 필딩은 HTTP의 주요 저자 중 한명
- 웹의 장점을 최대한 활용할 수 있는 아키텍처로 REST를 발표
- REST의 구성
 - 자원(Resource) - URI
 - 행위(Verb) - HTTP Method
 - 표현(Representations)

REST API 디자인 가이드

REST API 중심규칙

- 규칙1: URI는 정보의 자원을 표현(명사를 사용)

- URL에 행위에 대한 표현이 들어가면 안됨

GET /members/delete/1

- 규칙2: 자원에 대한 행위는 HTTP Method로 표현

DELETE /members/1

GET /members/show/1

GET /members/1

GET /members/insert/2

POST /members/2

REST API 디자인 가이드

REST API 중심규칙

Method	역할
GET	리소스 조회
POST	리소스 생성
PUT	리소스 수정
DELETE	리소스 삭제

REST API 디자인 가이드

URI 설계 시 주의할 점

- 슬래쉬(/) 구분자는 계층 관계를 나타내는데 사용

<http://restapi.example.com/houses/apartment>

<http://restap.example.com/animals/mammals/whales>

- 하이픈(-)은 URIGA독성을 높이는데 사용
- 언더스코어(_)는 URI에 사용하지 않음
- URI 경로에는 소문자가 적합

REST API 디자인 가이드

URI 설계 시 주의할 점

- 파일 확장자는 URI에 포함시키지 않음

`http://restapi.example.com/members/345/photo.jpg`

`GET / members/345/photo`

`HTTP/1.1 Host: restapi.example.com Accept: image/jpg`

- 리소스 간의 관계 표현

`has - GET /students/1001/cars`

`likes - GET /students/1001/likes/cars`

REST API 디자인 가이드

URI 설계 시 주의할 점

- Collection과 Document

`http://resapi.example.com/sports/soccer`

`http://restapi.example.com/sports/soccer/players/1001`

- 필터를 위해서 쿼리 파라미터를 사용

`http://api.sample.com/students?major=컴퓨터공학&grade=1`

`http://resapi.example.com/books?page=1&size=10&q=react`